Dungeon RPG — Code Explanations

1. Imports and Game Setup

Purpose: Import the Python modules needed to run the game.

The program imports three modules:

- `argparse`: Allows the user to choose game settings through the terminal.
- `json`: Allows the program to save and load game information using JSON files.
- `random`: Allows the game to randomly select enemies, encounters, rewards, and ability effects.

These modules provide the additional functionality needed for the game.

2. Class Information and Enemy Information

Purpose: Store the statistics and information for all playable classes and enemies.

The `CLASS_INFO` dictionary stores each character class's health, damage, mana, description, and special ability.

The `ENEMY_TYPES` list stores the different enemies that can appear in the dungeon, including their health, damage, and type.

The `DIFFICULTY_MULTIPLIERS` dictionary stores the difficulty settings used to adjust enemy strength.

These collections make it easier to access and change game information without repeatedly writing the same values throughout the program.

3. Player Class

Purpose: Create and manage the two players in the game.

The `Player` class stores information about each player, including their name, character class, health, damage, mana, and inventory.

It contains several methods:

- `is_alive()`: Checks whether the player's health is above zero.
- `take_damage()`: Reduces the player's health.
- `heal()`: Restores the player's health without exceeding their maximum health.
- `restore_mana()`: Restores the player's mana.
- `to_dict()`: Converts the player's information into a dictionary for saving.
- `from_dict()`: Recreates a Player object using information from a saved game.

Each player is created as an individual object, allowing both players to have their own statistics and inventory.

4. Enemy Class

Purpose: Create and manage enemies during combat.

The `Enemy` class stores the enemy's name, type, health, and damage.

It also stores temporary combat effects, such as being stunned or affected by decay damage.

Its methods include:

- `is_alive()`: Checks whether the enemy still has health.
- `take_damage()`: Reduces the enemy's health.

This class allows the program to create different enemies using the same structure while keeping track of their individual statistics and combat effects.

5. Dungeon Class

Purpose: Control dungeon progression and generate random encounters.

The `Dungeon` class stores the current floor and selected difficulty.

Its `create_encounter()` method determines whether an enemy appears. There is a 75% chance of an enemy encounter and a 25% chance of finding an empty floor.

The enemies that can appear depend on the current floor:

- Floors 1–2: The first three enemy types can appear.
- Floors 3–4: The first five enemy types can appear.
- Floors 5–10: All enemy types can appear.

Enemy health increases by two for each additional floor, while enemy damage increases every two floors. The selected difficulty multiplier is then applied.

The `advance()` method increases the current floor by one.

6. Save Manager Class

Purpose: Save and load the player's progress using JSON files.

The `SaveManager` class contains two main methods:

- `save_game()`: Collects the current floor, difficulty, and both players' information, then writes it into a JSON file.
- `load_game()`: Reads the JSON file and recreates the Player objects and Dungeon object.

This allows players to save their progress and continue their game later without losing their character information, inventory, or dungeon progression.

The program saves between floors rather than during an active battle.

7. Game Class and Player Creation

Purpose: Manage the overall game and create the two playable characters.

The `Game` class brings the different parts of the program together. It stores the players, dungeon, and save filename.

The `show_class_information()` method displays the statistics and abilities of all five character classes.

The `create_players()` method asks two players for their names and allows each person to select a character class.

The program checks that the names and class selections are valid before creating the Player objects.

8. Main Menu

Purpose: Allow players to navigate the game.

The `main_menu()` method displays the main menu with four options:

1. New Game
2. Load Game
3. Class Information

4. Exit

Selecting New Game creates two new players and starts the dungeon from floor one.

Selecting Load Game allows the user to enter a save filename and attempts to restore the saved game.

Selecting Class Information displays the available character classes and their statistics.

Selecting Exit closes the program.

The menu uses a loop so players can continue choosing options until they decide to exit.

9. Inventory and Class Abilities

Purpose: Allow players to use items and their character's special abilities.

The `use_item()` method displays the player's inventory and allows them to use Health Potions or Mana Potions.

- Health Potions restore 15 health.
- Mana Potions restore 12 mana.

The `class_ability()` method handles the special abilities for each character class. All abilities cost eight mana.

Each class has a different ability:

- Moth Keeper: Applies decay damage to the enemy for two to three turns.
- Bell Ringer: Stuns the enemy for one turn but causes both teammates to take three damage.
- Chef: Creates a meal with an 80% chance of a beneficial result and a 20% chance of backfiring.
- Fisherman: Attacks the enemy using a giant bass, dealing three times their normal attack damage.
- Librarian: Randomly receives two ability choices from the other four classes and must select one.

These methods give players different ways to manage their resources and approach combat.

10. Player Turns and Enemy Turns

Purpose: Control the actions that occur during battles.

The `player_turn()` method displays the player's current health, mana, and the enemy's health.

Players can choose between four actions:

1. Attack
2. Class Ability
3. Inventory
4. View Party Status

The `enemy_turn()` method controls the enemy's actions.

If the enemy is stunned, it loses its opportunity to attack. Otherwise, it randomly selects a living player and attacks them.

The enemy's decay effect is also processed during its turn.

These methods separate player actions from enemy actions and help organize the combat system.

11. Combat, Rewards, and Dungeon Exploration

Purpose: Manage battles, distribute rewards, and progress through the dungeon.

The `fight()` method controls the entire battle.

The combat order is:

1. Player One takes their turn.
2. Player Two takes their turn.
3. The enemy takes its turn.

This continues until the enemy is defeated or both players are defeated.

After winning a battle, the `victory_rewards()` method restores four mana to each player and randomly determines whether the players receive a Health Potion, Mana Potion, or no item.

The `show_party()` method displays both players' current statistics and inventories.

Finally, the `explore_dungeon()` method manages progression through all ten floors. After completing a floor, players can continue to the next floor or save their progress and return to the main menu.

Completing all ten floors results in victory.

12. Argparse and Program Startup

Purpose: Allow the game to be started with different settings through the terminal.

The `parse_arguments()` function uses `argparse` to provide three command-line options:

- `--difficulty`: Selects easy, normal, or hard difficulty.
- `--save`: Selects the filename used to save the game.
- `--load`: Tells the program to load the selected save file when starting.

The `main()` function is responsible for starting the program.

It checks whether the user requested a saved game. If so, it attempts to load the save file. Otherwise, it creates a new game using the selected difficulty.

The final `if __name__ == "__main__":` statement ensures that the program starts by calling `main()` when the Python file is run directly.

Overall Project Summary

The Dungeon RPG is a two-player, turn-based adventure game where players select character classes, explore a ten-floor dungeon, fight randomly generated enemies, and collect items.

The rewritten program uses object-oriented programming to organize players, enemies, dungeon progression, saving, and overall game management into separate classes.

It also includes command-line arguments through `argparse`, JSON saving and loading, and a terminal-based menu system.

These additions make the program more organized and easier to maintain while keeping the original two-player gameplay and character abilities.